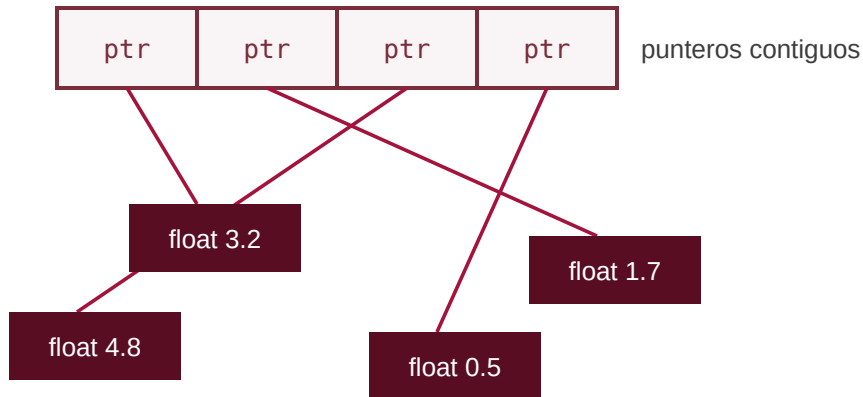


Por qué un ndarray es más rápido: la diferencia está en la memoria

list — arreglo de referencias a objetos dispersos

```
v = [3.2, 1.7, 4.8, 0.5]
```



Cada número es un objeto completo ($\approx 24\text{--}32$ bytes)
Sumar dos listas obliga a seguir un puntero por elemento,
comprobar el tipo y crear un objeto nuevo con el resultado.

El bucle ocurre en el intérprete de Python.

ndarray — un bloque contiguo y homogéneo

```
a = np.array([3.2, 1.7, 4.8, 0.5])
```

3.2	1.7	4.8	0.5
8 bytes	8 bytes	8 bytes	8 bytes
32 bytes seguidos, dtype = float64			

Metadatos, aparte del bloque

```
shape = (4,)  
dtype = float64  
strides = (8,)
```

El tipo se conoce una sola vez, para todo el bloque.

**El bucle ocurre en código C compilado, sobre datos
que caben juntos en la caché del procesador.**

Consecuencia: la lista es flexible y la usamos para construir; el ndarray es rígido y lo usamos para calcular.